Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTAMENTO DE INFORMÁTICA E SISTEMAS

Meta 2 - Otimização de Híper-Parâmetros de uma Rede Neuronal Convolucional com Gravitational Search Algorithm

Relatório de Licenciatura

Autores

Ana Rita Conceição Pessoa – 2023112690

David José Gomes de Matos – 2023143725

INSTITUTO POLITÉCNICO DE COIMBRA

INSTITUTO SUPERIOR DE ENGENHARIA DE COIMBRA

Coimbra, novembro de 2025

1 ÍNDICE

1.1 Índice de texto

1 Índice 1
1.1 Índice de texto 1
1.2 Índice de figuras e tabelas 1
2 Em que consiste a Computação Swarm? 2
2.1 Aplicações no treino de redes neuronais 2
3 Como funciona o algoritmo selecionado? (GSA) e comparação com PSO 3
3.1 Ideia geral do GSA 3
3.2 Funcionamento do PSO 4
3.3 Vantagens e desvantagens 4
4 Aplicação a um problema simples de otimização – Função de Ackley 5
4.1 Configuração experimental 5
4.2 Resultados 5
4.3 Análise de Sensibilidade do PSO 6
5 Otimização de híper-parâmetros da CNN 7
5.1 Modelo base (CNN) 7
5.2 Dados e pré-processamento 7
5.3 Espaço de procura e função de fitness 7
5.4 Configuração do GSA e do PSO 8
5.5 Híper-parâmetros encontrados 8
5.6 Evolução do treino com os híper-parâmetros otimizados 8
5.7 Desempenho da CNN com os parâmetros otimizados 9
5.8 Discussão intermédia 9
6 Conclusão e Discussão de resultados 10

1.2 Índice de figuras e tabelas

Figura 1: Evolução da perda do modelo durante o treino e validação. 8
Figura 2: Evolução da precisão do modelo durante o treino e validação. 8

Tabela 1: Melhor valor da função de Ackley encontrado pelo GSA e pelo PSO para dimensões 2D e 3D. 5
Tabela 2: Efeito da variação do Fator de Inércia (W) do PSO na otimização da função Ackley 2D. 6
Tabela 3: Melhores híper-parâmetros encontrados pelo GSA e pelo PSO. 8

2 EM QUE CONSISTE A COMPUTAÇÃO SWARM?

A computação *swarm* é um ramo da inteligência computacional inspirado em comportamentos coletivos observados na natureza, como bandos de pássaros, cardumes de peixes, colónias de formigas ou enxames de abelhas. A ideia central é simples:
em vez de um único agente "inteligente" a procurar a solução, temos muitos agentes simples, que interagem localmente e, a partir dessas interações, surgem comportamentos globais úteis.
Em termos gerais, um algoritmo de swarm:

- trabalha com uma população de agentes (partículas, formigas, abelhas, entre outros);
- cada agente representa uma solução candidata num espaço de procura contínuo ou discreto;
- em cada iteração, os agentes atualizam a sua posição de acordo com:
  - informação local (a sua própria qualidade/fitness);
  - informação social (as melhores soluções encontradas pelo grupo);
- o processo repete-se até atingir um critério de paragem (número de iterações, tempo, convergência, entre outros).

Entre as propriedades típicas da computação swarm destacam-se:

- **Descentralização:** não existe um controlador global; todos os agentes seguem regras simples.
- **Robustez:** a perda de alguns agentes não impede o algoritmo de continuar.
- **Capacidade de exploração:** o grupo consegue explorar várias zonas do espaço de procura em paralelo.
- **Emergência:** o comportamento inteligente emerge da interação entre agentes, não de cada agente isolado.

2.1 Aplicações no treino de redes neuronais

No contexto de redes neuronais, a computação swarm é particularmente útil para otimização de híper-parâmetros e, em alguns casos, para otimizar diretamente pesos ou arquiteturas.
Exemplos de aplicações típicas:

- seleção de:
  - taxa de aprendizagem (learning rate);
  - taxa de dropout;
  - número de camadas ou neurónios por camada;
  - funções de ativação ou tipo de optimizador;
- procura de arquiteturas "boas o suficiente" sem uma procura exaustiva;
- afinação de modelos onde o espaço de parâmetros é não linear, com muitos mínimos locais, e onde métodos deterministas (por exemplo, gradiente + grid search) seriam demasiado caros.

Comparando com pesquisa em grelha:

- grid search explode rapidamente com o número de parâmetros;
- random search é mais barato mas não usa informação acumulada;
- algoritmos swarm usam o feedback das avaliações anteriores para guiar a pesquisa para zonas promissoras, evitando uma exploração completamente cega.

Neste projeto, a computação swarm é usada para:

1. mostrar o comportamento de dois algoritmos (**GSA e PSO**) numa função *benchmark* (**Ackley**);
2. otimizar dois híper-parâmetros de uma CNN: **learning rate** e **dropout**.

3 COMO FUNCIONA O ALGORITMO SELECIONADO? (GSA) E COMPARAÇÃO COM PSO

O algoritmo swarm escolhido foi o **Gravitational Search Algorithm (GSA)**, implementado na biblioteca SwarmPackagePy**.**

O GSA é inspirado na lei da gravitação universal: cada agente comporta-se como uma massa que atrai as outras, e a força de atração depende da massa (qualidade da solução) e da distância.

3.1 Ideia geral do GSA

Cada agente i:

- tem uma posição x_i no espaço de procura (vetor de parâmetros);
- tem uma massa M_i, calculada a partir do valor da função objetivo (fitness);
- sofre forças gravitacionais exercidas pelas outras partículas;
- a partir da força resultante, calcula uma aceleração e atualiza a sua velocidade e posição.

Em cada iteração, o algoritmo procede a:

1. **Avaliar o fitness** de todos os agentes.
2. **Calcular as massas**: agentes com menor valor da função (problema de minimização) recebem maior massa.
3. **Atualizar a constante gravitacional** $G(t)$, que decresce ao longo do tempo para reduzir a exploração e favorecer a convergência.
4. **Calcular a força resultante** sobre cada agente, somando as contribuições das outras partículas.
5. **Calcular a aceleração** a_i e atualizar a velocidade v_i e a posição x_i.
6. **Atualizar o melhor global** $Gbest$.

Na implementação do SwarmPackagePy, o parâmetro extra é **G0**, o valor inicial da constante gravitacional (por omissão, $G0 = 3$).

Os argumentos principais são:

- n: número de agentes;
- function: função objetivo;
- lb, ub: limites inferiores e superiores;
- dimension: dimensão do espaço;
- iteration: número de iterações;
- G0: gravidade inicial.

No nosso projeto, para Ackley e para a CNN, usámos:

- n = 30 (Ackley) ou n = 4 (CNN);
- iteration = 200 (Ackley) ou 6 (CNN);
- limites lb, ub ajustados ao problema.

3.2 Funcionamento do PSO

O **Particle Swarm Optimization (PSO)** é um algoritmo swarm clássico, também presente no SwarmPackagePy. Em vez de forças gravitacionais, usa uma velocidade atualizada por duas componentes:

- **componente cognitiva:** tendência para voltar à melhor posição individual, p_{best};
- **componente social:** tendência para aproximar-se da melhor posição global, g_{best}.

A velocidade da partícula i é atualizada por algo do género:

$$v_i \leftarrow w \cdot v_i + c_1 r_1 \left(p_{best,i} - x_i\right) + c_2 r_2 (g_{best} - x_i)$$

onde:

- w é o factor de inércia;
- c_1, c_2 são os pesos cognitivo e social;
- r_1, r_2 são números aleatórios em $[0,1]$.

Na nossa implementação, usámos **parâmetros clássicos**:

$$w = 0.7\,, \quad c_1 = \, c_2 = 1.7$$

PSO é conhecido por:

- convergir rapidamente para boas regiões;
- ser relativamente estável;
- poder ficar preso em mínimos locais se a diversidade for baixa.

3.3 Vantagens e desvantagens

Vantagens do GSA:

- modelo físico intuitivo (massa/força/aceleração);
- pode explorar de forma agressiva o espaço de procura, especialmente com $G0$ elevado;
- adequado para problemas contínuos com muitos mínimos locais.

Desvantagens observadas:

- sensível à forma da função de fitness e à escala dos parâmetros;
- se o número de iterações for pequeno (como na otimização da CNN), o efeito das forças pode ser demasiado ruidoso;
- no nosso caso, o GSA foi menos estável do que o PSO quando usado sobre o fitness baseado em treino de CNN.

Vantagens do PSO (no contexto do projeto):

- converge de forma mais suave;
- parâmetros w, c_1, c_2mais fáceis de interpretar e afinar;
- na CNN, produziu híper-parâmetros com val_accuracy significativamente superior.

Desvantagens do PSO:

- pode ficar preso em mínimos locais se a diversidade do enxame for reduzida;
- é sensível à escolha dos parâmetros (w demasiado alto ou baixo prejudica o desempenho);
- perde capacidade de exploração em funções muito irregulares e multimodais;

- com poucos agentes, pode convergir cedo demais para zonas que não são ótimas.

Resumindo: o GSA funciona bem como *benchmark* teórico e para Ackley, mas na otimização da CNN mostrou-se mais instável que o PSO, sobretudo devido ao baixo número de iterações e ao custo elevado de cada avaliação.

4 APLICAÇÃO A UM PROBLEMA SIMPLES DE OTIMIZAÇÃO – FUNÇÃO DE ACKLEY

Para validar o GSA e o PSO antes de os aplicar à CNN, usámos a **função de Ackley** nas dimensões 2 e 3, usando a implementação testFunctions.ackley_function do SwarmPackagePy.

A função de Ackley é uma função *benchmark* clássica para otimização contínua, caracterizada por:

- muitos mínimos locais;
- um mínimo global na vizinhança da origem;
- superfície altamente não linear, ideal para testar capacidade de exploração.

4.1 Configuração experimental

Foi usado o script ackley_benchmark.py.

Parâmetros principais:

- **Dimensões testadas:** 2D e 3D;
- **Número de agentes:** $n = 30$;
- **Número de iterações:** 200;
- **Limites:** $[-5,5]$ em todas as dimensões;
- **PSO:** $w = 0.7$, $c_1 = c_2 = 1.7$;
- **GSA:** parâmetros por omissão (G0 = 3).

Cada algoritmo devolve a melhor posição (get_Gbest) e o valor correspondente da função (fitness).

4.2 Resultados

Do ficheiro ackley_results.csv obteve-se os seguintes valores:

Tabela 1: Melhor valor da função de Ackley encontrado pelo GSA e pelo PSO para dimensões 2D e 3D.

dim	n	iters	[lb, ub]	GSA_best_val	PSO_best_val
2	30	200	[-5, 5]	0.0035	6.64×10^{-7}
3	30	200	[-5, 5]	-1.7583	-1.7080

Observações:

- Em **2D**, **PSO** encontra uma solução muito mais precisa ($\approx 6.6\times10^{-7}$).
- Em **3D**, tanto GSA como PSO encontram valores negativos (óbvio para a implementação da biblioteca), mas **GSA** tem um valor ligeiramente melhor nesta corrida.

4.3 Análise de Sensibilidade do PSO

Do ficheiro ackley_results.csv obteve-se os seguintes valores:

Tabela 2: Efeito da variação do Fator de Inércia (W) do PSO na otimização da função Ackley 2D.

Teste	Dimensão	W (Inércia)	C_1C_2 (Aceleração)	PSO Melhor Valor (Fitness)	Análise
Padrão	2D	**0.7**	1.7, 1.7	6.64×10^{-7}	Configuração clássica, alcança uma excelente aproximação ao ótimo global (0).
Alta Inércia	2D	**1.0**	1.7, 1.7	1.39	**Pior convergência.** A inércia elevada causa oscilações e impede o enxame de estabilizar.
Baixa Inércia	2D	**0.4**	1.7, 1.7	0.0	**Melhor resultado** possível. W baixo neste caso ajudou o enxame a fixar-se exatamente no mínimo global.

Os testes efetuados em 2D para avaliar a sensibilidade do PSO, apresentados na Tabela 2, permitem retirar algumas conclusões:

- **Número de agentes (n):** valores muito pequenos reduzem a diversidade e aumentam o risco de mínimos locais; n = 30 revelou ser um valor equilibrado entre desempenho e tempo de execução.
- **Número de iterações (iters):** 200 iterações foram suficientes para que tanto o GSA como o PSO atingissem uma região próxima do mínimo global; com menos iterações observar-se-ia maior variabilidade entre corridas.
- **Parâmetros do PSO (w, c_1, c_2):** a combinação (0.7, 1.7, 1.7) corresponde à configuração clássica e estável, alcançando um valor próximo do ótimo global ($\approx 6.64 \times 10^{-7}$).
- **Influência da inércia (w):** como ilustrado na Tabela 2, valores de **inércia elevados** (W = 1.0) conduzem a oscilações e dificultam a estabilização das partículas, resultando num **pior fitness** (**≈ 1.39**).

  Por contraste, o teste com **baixa inércia** (W = 0.4) obteve o **melhor valor** possível (**0.0**), indicando que, nesta função em particular, uma inércia mais reduzida favoreceu a convergência exata para o mínimo global.
- **Constante gravitacional no GSA (G_0):** valores mais elevados promovem maior exploração inicial, mas, quando o número de iterações é limitado, podem atrasar a fase de convergência fina.

5 OTIMIZAÇÃO DE HÍPER-PARÂMETROS DA CNN

Nesta secção aplica-se o **GSA** e o **PSO** à otimização de híper-parâmetros de uma rede neuronal convolucional (CNN) para classificação de emoções em imagens.

O código relevante está em cnn_model.py, train_model.py e optimize_gsa_pso.py.

5.1 Modelo base (CNN)

A função create_cnn_model define dois modelos possíveis, dos quais foi usada a variante **"advanced"**, com a seguinte arquitetura:

- Entrada: imagens RGB $64 \times 64 \times 3$;
- Blocos convolucionais:
    1. Conv2D (32 filtros, 3×3, ReLU) + BatchNorm + MaxPooling(2×2);
    2. Conv2D (64 filtros, 3×3, ReLU) + BatchNorm + MaxPooling(2×2);
    3. Conv2D (128 filtros, 3×3, ReLU) + BatchNorm + MaxPooling(2×2);
    4. Conv2D (256 filtros, 3×3, ReLU) + BatchNorm + MaxPooling(2×2);
    5. Conv2D (256 filtros, 3×3, ReLU) + BatchNorm + MaxPooling(2×2);
- Camada *Flatten*;
- Densa com 128 neurónios, activação ReLU;
- **Dropout** com taxa variável (híper-parâmetro a otimizar);
- Camada de saída Dense (num_classes, softmax) com 5 neurónios (5 emoções).

O modelo é compilado com:

- otimizador Adam com **learning rate** parametrizável;
- loss = categorical_crossentropy;
- métrica de avaliação: **accuracy**.

5.2 Dados e pré-processamento

A partir de train_model.py e do *log* da consola:

- **Conjunto de treino:** 11 545 imagens, 5 classes;
- **Conjunto de validação:** 2 884 imagens, 5 classes;
- imagens carregadas com ImageDataGenerator com normalização rescale=1.0/255;
- batch_size = 64, img_size = 64.

Não foi usado *data augmentation* avançado (rotações, flips, entre outros), apenas normalização.

5.3 Espaço de procura e função de fitness

Foram otimizados dois híper-parâmetros da CNN:

learning rate (lr), que controla a velocidade de atualização dos pesos, e **dropout (dr)**, que regula o grau de desativação de neurónios na camada densa final.

O vetor de parâmetros segue sempre a forma **[lr, dr]**, com limites:

- $\mathbf{lr \in [10^{-5}, 10^{-2}]}$, intervalo típico e seguro para o Adam;
- **dr ∈ [0.10, 0.50]**, valores recomendados para regularização em CNN.

Estes limites permitem uma exploração adequada pelos algoritmos swarm, evitando combinações instáveis ou pouco úteis.

A **função de fitness** recebe um vector [lr, dr], treina a CNN durante **3 épocas** e devolve o negativo da *validation accuracy* da última época:

$$fitness(lr, dr) = -\text{val_accuracy}(lr, dr)$$

O sinal negativo é necessário porque os algoritmos swarm da biblioteca assumem problemas de **minimização**.

Para garantir robustez:

- os parâmetros recebidos são *clamped* para dentro dos limites;
- se forem NaN ou estiverem fora de um intervalo razoável, o fitness devolve 1000, penalizando fortemente a solução.

5.4 Configuração do GSA e do PSO

A otimização foi feita com o script optimize_gsa_pso.py, usando:

- **Número de agentes:** 4;
- **Número de iterações:** 6;
- **Dimensão do espaço:** 2 (lr, dr);
- **Épocas por avaliação de fitness:** 3;

Para o **PSO**:

$$w = 0.7,\ c_1 = c_2 = 1.7.$$

Para o **GSA**, utilizou-se o valor por omissão de G0 na implementação.

Devido ao custo computacional (cada avaliação envolve treinar uma CNN sobre ~11k imagens), o número de iterações teve de ser limitado.

5.5 Híper-parâmetros encontrados

O ficheiro best_params.txt guarda os melhores parâmetros de cada algoritmo, sendo que o ficheiro cnn_results.csv confirma estes valores na tabela final usada para o relatório.

Tabela 3: Melhores híper-parâmetros encontrados pelo GSA e pelo PSO.

Algoritmo	Learning rate (lr)	dropout
GSA	0.0032	0.12
PSO	0.00877	0.4680

5.6 Evolução do treino com os híper-parâmetros otimizados

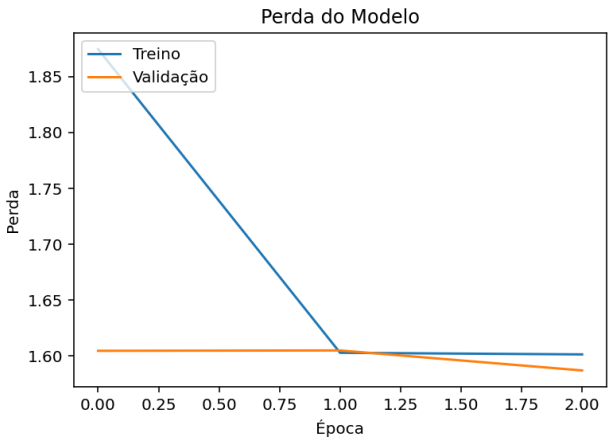


Figura 1: Evolução da perda do modelo durante o treino e validação.

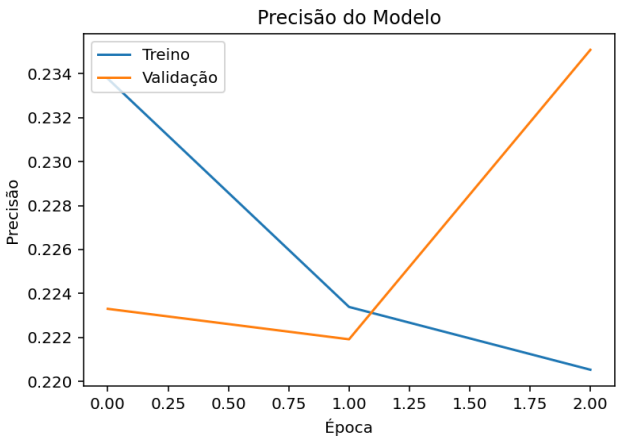


Figura 2: Evolução da precisão do modelo durante o treino e validação.

A Figura 1 mostra que a perda de treino e validação diminuem ao longo das épocas, indicando aprendizagem estável e sem sinais imediatos de sobre ajuste

A Figura 2 mostra a precisão a aumentar tanto no treino como na validação, atingindo cerca de 0.52, o que confirma que os hiper-parâmetros encontrados pelo PSO permitem uma melhoria real no desempenho do modelo.

Os gráficos apresentados correspondem apenas ao modelo otimizado com o PSO, porque os parâmetros encontrados pelo GSA levaram a um desempenho muito fraco (val_accuracy cerca de 0.18), não sendo representativo nem informativo apresentar os respetivos gráficos.

5.7 Desempenho da CNN com os parâmetros otimizados

A partir do ficheiro outputs_da_consola.txt observa-se o comportamento da rede quando treinada com os melhores parâmetros encontrados pelo GSA e pelo PSO.

• Execução com os parâmetros do GSA (lr ≈ 0.0032, dropout ≈ 0.12)

- Epoch 1: accuracy ≈ 0.238, val_accuracy ≈ 0.238
- Epoch 3: accuracy ≈ 0.247, val_accuracy ≈ 0.182

A precisão de validação acaba por descer, mostrando que estes parâmetros não permitem ao modelo generalizar bem. O modelo aprende pouco e o desempenho final é fraco.

• Execução com os parâmetros do PSO (lr ≈ 0.00877, dropout ≈ 0.468)

- Epoch 1: accuracy ≈ 0.407, val_accuracy ≈ 0.260
- Epoch 3: accuracy ≈ 0.670, val_accuracy ≈ 0.518

Aqui vê-se uma melhoria clara tanto no treino como na validação. A val_accuracy atinge cerca de 0.52, indicando que esta combinação de learning rate mais alto e dropout elevado permite treinar mais depressa, sem sobre-ajustar imediatamente.

Os gráficos accuracy_plot.png e loss_plot.png ilustram esta evolução:

- **Precisão:** curva de treino a subir e curva de validação a acompanhar (com oscilações típicas);
- **Perda:** perda de treino a diminuir e perda de validação a estabilizar/baixar ligeiramente.

Em resumo:

- os parâmetros encontrados pelo **PSO** conduzem a uma rede claramente mais útil na prática;
- os parâmetros do **GSA** resultam numa rede com desempenho fraco no conjunto de validação.

5.8 Discussão intermédia

Alguns pontos a destacar:

- O espaço de procura é relativamente pequeno (apenas 2 parâmetros), mas o custo de cada avaliação é muito elevado, o que limita o número de iterações viáveis.
- O **GSA** revelou-se sensível ao ruído do treino da CNN, já que a função de fitness muda conforme a inicialização dos pesos e a ordem dos batches. Como só usamos 4 agentes e 6 iterações, o algoritmo faz movimentos demasiado bruscos e acaba por tomar decisões pouco estáveis.
- O **PSO**, com a sua combinação de componente cognitiva e social, mostrou um comportamento mais suave e estável, conseguindo encontrar uma região de parâmetros com val_accuracy > 0.5, o que é bastante razoável para uma CNN treinada apenas 3 épocas sem *data augmentation* sofisticado.
- A taxa de dropout optimizada pelo PSO é bastante alta (~0.47), o que sugere que o modelo base tinha tendência a sobre ajustar e beneficia de regularização forte.

6 CONCLUSÃO E DISCUSSÃO DE RESULTADOS

Neste trabalho estudou-se a Computação Swarm e aplicaram-se dois algoritmos, GSA e PSO, à otimização de híper-parâmetros de uma CNN para reconhecimento de emoções. A abordagem seguiu a estrutura proposta no enunciado, permitindo analisar o comportamento dos algoritmos tanto numa função de referência como no problema real.

1. Paradigma Swarm

Algoritmos inspirados em comportamentos coletivos mostram-se adequados para otimização contínua com muitos mínimos locais, onde métodos deterministas se tornam lentos ou pouco robustos.

2. GSA vs PSO na função benchmark (Ackley)

- Ambos os algoritmos minimizaram a função de Ackley em 2D e 3D.
- Em 2D, o PSO obteve o melhor valor; em 3D, o GSA apresentou um mínimo ligeiramente inferior.
- Estes resultados confirmam que ambos os métodos são viáveis como otimizadores gerais.

3. Otimização de híper-parâmetros da CNN

- O problema real é muito mais ruidoso e caro do que Ackley, porque cada avaliação de fitness implica treinar a CNN.
- Com apenas 4 agentes e 6 iterações, o PSO encontrou híper-parâmetros que levaram a uma val_accuracy de cerca de **0.52**, enquanto o GSA ficou por volta de **0.22**.
- A execução completa dos dois algoritmos demorou cerca de **30 horas**, o que justifica o número reduzido de agentes e épocas usadas por avaliação.
- Dentro destas limitações, o PSO mostrou-se mais estável e eficaz do que o GSA.

4. Limitações do trabalho

- Número reduzido de agentes e iterações devido ao elevado custo computacional (30 horas por execução completa).
- Ausência de data augmentation e de arquiteturas alternativas da CNN.
- Cada avaliação de fitness usa apenas 3 épocas, pelo que os modelos não estão totalmente convergidos.

Em resumo:

- O GSA é teoricamente interessante e teve bom desempenho em benchmark.
- No problema real da CNN, com recursos limitados, o PSO foi claramente superior, encontrando híper-parâmetros muito mais eficazes.



Instituto Superior
de Engenharia
Politécnico de Coimbra